From Content-Addressed Storage to Governed Semantic Project Lineage

Prior Art, Architectural Differentiation, and a Novelty Thesis for an AI-Operable Project Substrate

Exact bytes are not yet a project, and a project history is not yet an authority model.

Author
Adam Ableman
Constitutional Computing
Contact: ableman.research@gmail.com

Abstract

This paper examines whether the proposed Semantic Project Lineage System represents a genuinely new computing architecture or primarily recombines established mechanisms.

The proposed system is a local-first, content-addressed, transtemporal substrate for storing, reconstructing, governing, and navigating evolving digital projects. It represents projects through exact content objects, persistent file-state lineages, version-scoped path maps, provenance and authority relationships, and reconstructible external projections such as ZIP archives and working directories.

The system is designed in particular for human–AI project workflows. An AI agent may repeatedly emit a complete project artifact under the same visible filename. Before that artifact replaces the prior visible version, the substrate independently verifies continuity, identifies every predecessor-file disposition, refuses unexplained losses, commits newly admitted content, and preserves exact reconstruction of prior states.

The review finds substantial prior art for nearly every primitive involved:

- content-addressed immutable storage;
- global chunk deduplication;
- snapshot-based version control;
- mutable objects formed from immutable claims;
- semantic filesystem indexing;
- data provenance;
- reproducible derivation;
- versioned data repositories;
- staged agent filesystem mutation;
- Git-like agent memory and rollback.

However, the reviewed systems do not appear to combine these mechanisms into the same authority model and operating workflow.

The strongest differentiation is not a new hashing, chunking, graph, or delta algorithm. It is an architectural unification:

> The ordinary filesystem, project archive, and agent workspace become projections of a governed semantic-temporal graph whose authoritative state includes exact content, file ancestry, project membership, provenance, authority, causality, and reconstructible external representations.

The paper therefore concludes that the proposed system is best characterized as a **plausibly novel systems synthesis built from established primitives**. Its strongest research contribution would be a governed correspondence model connecting exact storage, semantic project history, safe agent publication, and ordinary filesystem usability. The novelty claim remains review-bounded: it describes the comparison set represented in this draft, not universal priority, patentability, or the absence of other systems. The paper also supplies a formal object model and proof obligations so that the synthesis can be evaluated as more than a feature list.

---

1. Scope and Method

This paper compares the proposed architecture with major relevant families of prior work:

1. Content-addressed archival storage
2. Deduplicating backup systems
3. Distributed content-addressed object graphs
4. Source version control
5. Versioned filesystem-tree deployment
6. Stable identities over immutable claims
7. Semantic filesystems
8. Data-versioning platforms
9. Provenance standards
10. Functional package management and reproducible derivation
11. Agent memory versioning
12. Agent-native filesystem safety

The comparison is architectural rather than exhaustive. It draws primarily from original papers, standards, and official documentation.

It does not establish:

- patent novelty;
- freedom to operate;
- the absence of unpublished systems;

- the absence of obscure academic prototypes;
- the absence of claims hidden within broader patents.

The proper claim is therefore limited:

> Within the systems and literature reviewed here, no single prior system was found that realizes the complete proposed combination.

---

2. The Proposed System

The Semantic Project Lineage System represents an evolving project using six principal layers.

2.1 Exact content custody

Every admitted byte sequence receives a cryptographic identity and a recoverable storage representation.

A content state may be stored as:

- a complete object;
- content-defined chunks;
- a delta from an earlier state;
- a compound recipe;
- a format-specific representation plus an exact residual;
- an opaque fallback object.

Exact reconstruction remains the final authority.

2.2 Persistent file-state lineage

A logical file may possess many content states over time.

The system records relationships such as:

- unchanged;
- modified;
- copied;
- renamed;
- removed;
- restored;
- derived;
- generated;
- split;
- merged.

Exact identity and inferred semantic relationship remain distinct.

2.3 Project-state maps

A project generation is a complete map from normalized paths to file states.

The project map, not an archive byte stream, becomes the primary semantic representation of the project.

2.4 Semantic and governance relationships

Files and project states may carry relationships concerning:

- role;
- dependency;
- provenance;
- causality;
- authority;
- work items;
- findings;
- tests;
- generation;
- derivation.

These relationships possess explicit evidence classes.

2.5 Exact external projections

A project state may be materialized as:

- a folder tree;
- a ZIP or TAR archive;
- an APK or JAR;
- a document container;
- a disk image;
- a working directory;
- a bounded AI context package.

Where historical exactness is promised, packaging metadata and residual objects preserve the exact original byte stream.

2.6 Governed publication

A proposed successor is not admitted merely because it is newer.

Every predecessor path must receive a disposition such as:

- unchanged;
- modified;
- renamed;
- copied;
- intentionally removed;

- replaced;
- generated differently;
- missing without authority.

An unexplained disappearance refuses publication.

This permits a human or agent to repeatedly emit `project-latest.zip` while the hidden substrate preserves all admitted generations.

2.7 Formal Correspondence Contract

The architecture distinguishes five objects that are often collapsed:

1. A content object $C_h = (h, b)$ binds bytes b to digest h.
2. A file state $F_i = (i, h, r, e)$ binds persistent file identity i to content h, typed relations r, and evidence class e.
3. A project state P_g is a complete generation-scoped map from normalized paths to file-state identifiers.
4. A projection $X_g = (P_g, q_g, z_g)$ binds a project state to packaging recipe q_g and any exact residual z_g needed to reconstruct the external representation.
5. An admission record A_g binds predecessor state, proposal, disposition map, authority basis, verifier result, and resulting standing.

For an external byte artifact B_g, exact replay requires

$$\text{reconstruct}(X_g) = B_g.$$

For every predecessor path, complete successor admission requires exactly one typed disposition:

$$\forall p \in \text{dom}(P_{g-1}), \quad |\{d \in D_g : d.\,\text{predecessor} = p\}| = 1.$$

An admissible disposition does not by itself prove that the successor is correct. It proves only that predecessor treatment is explicit and satisfies the configured admission policy. Likewise, a content hash proves byte identity under the chosen digest; it does not prove authority, semantic equivalence, causal provenance, or publication standing.

The architecture therefore carries separate commitments for state, admissibility, and meaning, with optional jurisdictional locality. In the terminology of *The Three Hash Rule*, these are H_s, H_c, H_m, and optionally H_j. Project admission may depend on all four, but no commitment substitutes for another.

2.8 Corpus Position

The system is a storage-and-project realization of several corpus-level abstractions:

Corpus construct	Project-lineage role
Semantic Custody	Keeps exact artifact, logical work, version family, concept, and standing distinct.
Constitutional Storage	Makes retention, deletion, reconstruction, and authority inspectable storage operations.
The Three Hash Rule	Separates object state, admissibility, meaning, and jurisdictional locality.
Governed Semantic Compilation	Compiles a bounded semantic and authority state into an admissible projection or refusal.
Reasoning by Way of Receipts	Makes admission, refusal, effect, verification, promotion, and revision inspectable as different records.
Governed Context and Memory	Produces bounded agent context from project state without treating model memory as project authority.

These are architectural relations, not claims that every corpus system is implemented by the project substrate or that the substrate grants standing to the records it stores.

---

3. Empirical Origin in Mosaic Lineage Archive

The architecture emerged from a concrete archive-lineage program rather than from an abstract design exercise.

Mosaic Lineage Archive established:

- exact archive-byte custody;
- deterministic package construction;
- reversible historical reconstruction;
- complete source disposition;
- explicit historical gaps;
- preservation of non-lineage objects;
- release-append automation;
- typed verification refusals;
- deletion-eligibility evidence;
- lineage-global temporal object reuse.

On the 16-release GSOL corpus, the program measured:

Representation	Size
Original ZIP collection	25.13 MB
Adjacent paired-delta representation	19.27 MB
Global temporal-object representation	11.83 MB

Every original ZIP was reconstructed exactly.

That experiment established that content could remain in lineage custody while disappearing from active project states and later return by reference.

The proposed system generalizes that result from ZIP-level objects to persistent logical files and arbitrary compound digital objects.

---

4. Content-Addressed Archival Storage

4.1 Venti

Venti is an important foundational predecessor.

It identifies immutable blocks by hashes of their contents, applies a write-once model, coalesces duplicate blocks, and serves as a building block for backup and snapshot systems.

Venti establishes several central principles reused by the proposed system:

- content identity independent of location;
- immutability;
- deduplication;
- durable historical storage;
- higher-level structures assembled from immutable blocks.

4.2 Differentiation

Venti does not itself model:

- persistent logical file identities;
- file-state ancestry;
- semantic roles;
- project completeness;
- AI successor admission;
- authority standing;
- exact reconstruction of arbitrary external packaging conventions as governed projections.

Venti supplies a storage substrate. The proposed system adds a project and governance model above such a substrate.

---

5. Deduplicating Backup Systems

5.1 Restic and Borg

Restic uses content-defined chunking to recognize unchanged regions even when bytes shift within a large file. This enables deduplication between files and across backups.

Borg similarly divides files into variable-length chunks and deduplicates across the entire repository. Its reuse does not depend on stable filenames, complete-file identity, timestamps, or fixed chunk positions.

These systems are close to the proposed storage mechanics because they already provide:

- whole-repository chunk reuse;
- cross-snapshot deduplication;
- efficient handling of slightly modified large files;
- exact file restoration;
- rename-insensitive storage efficiency.

5.2 Differentiation

Backup systems generally answer:

> Can the files from this snapshot be restored?

The proposed system additionally asks:

> What is the persistent identity of this file? How did its states evolve? Why did it change? What standing does this state possess? Did an agent have authority to remove it? Which external artifacts are projections of this state?

Backup metadata may record paths and snapshots, but it generally does not elevate semantic file ancestry, project-state admission, and authority correspondence into the central model.

---

6. Git and Snapshot-Based Version Control

6.1 Git's object model

Git is a content-addressed storage system built from blobs, trees, and commits.

Blobs represent content, trees map names to blobs or subtrees, and commits identify complete project snapshots along with parent commits and authorship information.

This provides major precedents for:

- content-addressed files;

- snapshot trees;
- project-state roots;
- parent relationships;
- branching;
- merging;
- reproducible historical checkout.

Git also uses packfiles and delta compression to reduce storage costs among related objects.

6.2 Rename and copy semantics

Git does not ordinarily store rename identity as an authoritative history edge. Rename and copy detection are diff-time operations that can be enabled, disabled, limited, and tuned.

This distinction is important.

Git authoritatively stores snapshots and content identities. A statement that one path was renamed into another is typically derived later from similarities between snapshots.

6.3 Differentiation

The proposed system overlaps Git heavily at the content and project-map layers, but differs in scope and authority.

It is intended to handle:

- arbitrary project artifacts, not only source repositories;
- exact historical container bytes;
- files not organized through explicit commits;
- complete archives emitted by AI sessions;
- same-name visible artifact replacement;
- unexplained omission refusal;
- preserved rejected and auxiliary objects;
- explicit authority classes;
- exact recursive container reconstruction;
- working directories as non-authoritative projections;
- download routing and local artifact custody.

Git is probably the nearest general predecessor to the project-state map. The proposed contribution lies in extending that concept beyond source history into governed project custody and agent publication.

---

7. OSTree and Versioned Filesystem Trees

OSTree describes itself as a userspace content-addressed versioning filesystem inspired by Git. It stores commits, directory trees, directory metadata, content objects, and extended attributes, and is designed for atomic deployment of complete filesystem trees.

OSTree is particularly relevant because it demonstrates:

- content-addressed complete filesystem states;
- deployment as a projection of repository state;
- metadata-aware file objects;
- atomic upgrade and rollback;
- separation of content from deployment.

Differentiation

OSTree focuses on deployable operating-system trees and assumes that certain data may be regenerated from source. The proposed system instead emphasizes:

- general project artifacts;
- preservation rather than regeneration assumptions;
- semantic file lineages;
- exact external package reconstruction;
- AI-authored successor admission;
- project-specific causality and authority.

OSTree is a strong precedent for the claim that an ordinary filesystem can be a projection of a deeper content-addressed state.

It does not appear to implement the full semantic-governance layer proposed here.

---

8. IPFS and Generalized Merkle DAGs

IPFS combines content-addressed block storage with content-addressed links, producing a generalized Merkle DAG capable of supporting versioned filesystems and other persistent data structures.

IPFS establishes that arbitrary compound objects can be represented as graphs of immutable, hash-addressed components whose branches may share subobjects.

Differentiation

IPFS primarily addresses distributed naming, transport, persistence, and retrieval.

The proposed system adds a much more opinionated application model:

- persistent file identity;
- project completeness;
- semantic change classes;
- authority and evidence standing;
- exact packaging projections;
- agent publication verifiers;
- local workflow integration.

IPFS could plausibly serve as a distributed object backend for the system, but it does not supply the proposed project semantics by itself.

---

9. Perkeep and Mutable Objects from Immutable Claims

Perkeep is one of the closest conceptual predecessors.

At its lowest layer, it treats stored data as arbitrary bytes. Its upper layers define schemas for files, directories, static sets, signed claims, and permanodes. A permanode is an immutable anchor for a mutable object whose changing state is described by claims.

This closely resembles the proposed separation between:

- immutable content states;
- stable logical object identity;
- historical mutations;
- retention claims;
- documentary assertions.

Differentiation

Perkeep demonstrates that mutable objects can be represented through immutable historical claims, but it does not appear to center:

- complete project-state correspondence;
- file-state delta selection;
- exact historical archive packaging;
- AI omission courts;
- ratification standing;
- agent context compilation;
- same-name governed publication.

Perkeep may be the closest predecessor to the proposed persistent file identity model.

The proposed system's distinction is the integration of that model with project-level snapshots, exact reconstruction, semantic relations, and governed agent operation.

---

10. Semantic Filesystems

The 1991 Semantic File Systems work proposed automatically extracting file attributes through format-specific transducers and supporting associative access to files through virtual directories and content-derived queries.

This is direct prior art for:

- format-aware file interpretation;
- semantic metadata extraction;
- associative retrieval;
- filesystem views generated from meaning rather than only physical paths.

Differentiation

Traditional semantic filesystems primarily improve how stored files are found and organized.

They generally do not make semantic attributes part of an exact historical custody and project-publication mechanism.

The proposed system combines semantic indexing with:

- versioned file states;
- exact content ancestry;
- project maps;
- historical disappearance and restoration;
- governed inference standing;
- exact external reconstruction.

The semantic filesystem supplies meaning over files.

The proposed system supplies meaning, time, custody, authority, and reconstruction together.

---

11. Data Versioning: lakeFS and Pachyderm

lakeFS applies Git-like repositories, commits, branches, merges, and zero-copy branches to object-storage data. Branches initially point to existing commits rather than duplicating stored objects, and validations can be connected to commit or merge workflows.

Pachyderm versions data through repositories, branches, and commits, and connects data lineage to pipeline provenance. Its branch and commit relationships can capture dependencies between input and output datasets.

These systems establish:

- version control beyond source code;
- zero-copy branching;

- data-state commits;
- validation gates;
- pipeline provenance;
- atomic or governed publication patterns.

Differentiation

They are primarily oriented toward object stores, datasets, pipelines, analytics, and machine-learning workflows.

The proposed system is more granular and local-project-oriented:

- persistent identities for ordinary files;
- exact reconstruction of original user-facing containers;
- path-level successor completeness;
- recursive archive abstraction;
- Explorer and terminal workspace projection;
- stable visible artifact slots;
- human and AI local workflow.

These systems show that Git-like semantics can be applied to arbitrary data. They do not appear to provide the same exact file/container and operator-governance synthesis.

---

12. Provenance Standards

The W3C PROV family defines a general model for describing entities, activities, people, derivation, attribution, procedures, versioning, and provenance itself. Its purpose is to support interoperable provenance exchange and assessments of reliability and trustworthiness.

W3C PROV is strong prior art for the proposed provenance and causal graph.

The proposed relations:

```
generated_by
derived_from
associated_with_agent
introduced_by_work_item
governed_by_decision
```

can often be expressed or mapped into PROV concepts.

Differentiation

PROV is an information model, not a complete storage, publication, reconstruction, and agent filesystem.

The proposed system could adopt PROV-compatible representations while adding:

- content-addressed custody;
- exact file-state reconstruction;
- project-state admission;
- path correspondence;
- storage deduplication;
- workspace projection;
- enforcement.

The contribution is not a new definition of provenance. It is making provenance operational inside an exact project substrate.

---

13. Nix and Reproducible Derivation

Nix and functional package management treat build outputs as results of declared inputs and derivations. This supports isolation, reproducibility, rollback, and parallel coexistence of multiple package versions.

Recent large-scale work rebuilding historical Nix packages found high rebuildability and substantial–though incomplete–bitwise reproducibility across hundreds of thousands of packages.

Nix supplies powerful prior art for:

- derivation identity;
- immutable outputs;
- input-to-output causality;
- coexistence of historical versions;
- rollback;
- reproducible construction.

Differentiation

Nix is fundamentally derivation-centered: outputs should be recreated from declared build inputs.

The proposed system is custody-centered:

- it preserves exact outputs even when their production process is unavailable or irreproducible;
- it distinguishes generated derivations from externally supplied artifacts;
- it preserves documentary and rejected states;
- it models ordinary mutable project files and archives.

The proposed architecture would benefit from Nix-like derivation records, but it cannot rely solely on regeneration.

14. Emerging Agent Memory and Agent Filesystems

14.1 Git-Context-Controller

Git-Context-Controller structures an agent's memory as a persistent filesystem with commit, branch, merge, and context operations. It is intended to support long-horizon reasoning, recovery, architectural experimentation, and handoff between sessions or agents.

This is direct emerging prior art for:

- Git-like agent memory;
- persistent reasoning artifacts;
- branchable agent context;
- long-horizon continuity.

14.2 AgentGit

AgentGit applies commit, revert, and branch semantics to multi-agent workflow state, enabling agents to compare and explore alternative trajectories.

14.3 YoloFS

YoloFS argues that agent safety and autonomy should be supported by the filesystem itself. It uses staged mutations, snapshots, and progressive permissions so that agent effects remain reviewable and recoverable.

YoloFS is especially close to the proposed safe publication model:

- mutations are staged;
- effects remain reviewable;
- snapshots permit recovery;
- the filesystem mediates agent authority.

14.4 GitLake

The emerging GitLake design applies repository-wide commits, temporary branches, and atomic publication to an agent-first lakehouse. Agents work in isolated branches and publish only through final merge.

14.5 Differentiation

These systems show that agent state, context, data, and filesystem mutations are rapidly converging on version-control semantics.

However, their central objects differ:

- GCC versions agent context and reasoning artifacts.

- AgentGit versions agent workflow trajectories.
- YoloFS governs filesystem mutations and recovery.
- GitLake versions lakehouse data states.

The proposed system's central object is the **complete governed project**, connecting:

- exact arbitrary bytes;
- logical file histories;
- project maps;
- packaging projections;
- authority;
- semantic relationships;
- human workspace;
- agent publication.

This broader integration remains the principal differentiation.

---

15. Comparative Matrix

- **System:** Venti
  Exact content custody: Yes
  Versioned project tree: Client-defined
  Persistent logical identity: No
  Semantic relationships: No
  Provenance: No
  Agent-safe staging: No
  Exact external container replay: Client-defined
  Authority/completeness courts: No

- **System:** Borg/restic
  Exact content custody: Yes
  Versioned project tree: Snapshot-oriented
  Persistent logical identity: Weak/path metadata
  Semantic relationships: No
  Provenance: Limited
  Agent-safe staging: No
  Exact external container replay: File restore
  Authority/completeness courts: No

- **System:** Git
  Exact content custody: Yes
  Versioned project tree: Yes
  Persistent logical identity: Blob identity; path history inferred
  Semantic relationships: Limited
  Provenance: Commit metadata
  Agent-safe staging: Through workflow

Exact external container replay: Not arbitrary original containers
Authority/completeness courts: Hooks/policy, not intrinsic

- **System:** OSTree
  Exact content custody: Yes
  Versioned project tree: Yes
  Persistent logical identity: Content objects
  Semantic relationships: Limited
  Provenance: Commit metadata
  Agent-safe staging: Atomic deployment
  Exact external container replay: Filesystem-tree deployment
  Authority/completeness courts: Limited

- **System:** IPFS
  Exact content custody: Yes
  Versioned project tree: DAG-defined
  Persistent logical identity: Through higher layers
  Semantic relationships: Application-defined
  Provenance: Application-defined
  Agent-safe staging: No
  Exact external container replay: Application-defined
  Authority/completeness courts: No

- **System:** Perkeep
  Exact content custody: Yes
  Versioned project tree: Object claims
  Persistent logical identity: Yes, through permanodes
  Semantic relationships: Claim-defined
  Provenance: Signed claims
  Agent-safe staging: No
  Exact external container replay: Not central
  Authority/completeness courts: Claim authorization

- **System:** Semantic FS
  Exact content custody: Filesystem-dependent
  Versioned project tree: Not central
  Persistent logical identity: Usually path/object based
  Semantic relationships: Yes
  Provenance: Possible
  Agent-safe staging: No
  Exact external container replay: No
  Authority/completeness courts: No

- **System:** lakeFS
  Exact content custody: Object-storage based
  Versioned project tree: Dataset namespace
  Persistent logical identity: Object/path based
  Semantic relationships: Limited

Provenance: Commit lineage
Agent-safe staging: Branch/merge workflow
Exact external container replay: Object state
Authority/completeness courts: Hooks

- **System:** Pachyderm
  Exact content custody: Yes
  Versioned project tree: Data repositories
  Persistent logical identity: Data/path based
  Semantic relationships: Pipeline DAG
  Provenance: Strong
  Agent-safe staging: Transactional workflow
  Exact external container replay: Dataset state
  Authority/completeness courts: Pipeline policy

- **System:** Nix
  Exact content custody: Store-path exactness
  Versioned project tree: Package closure
  Persistent logical identity: Derivation/output identity
  Semantic relationships: Dependency graph
  Provenance: Strong derivation
  Agent-safe staging: Build isolation
  Exact external container replay: Reproduced output
  Authority/completeness courts: Build policy

- **System:** GCC/AgentGit
  Exact content custody: Artifact/context dependent
  Versioned project tree: Agent-state tree
  Persistent logical identity: Agent artifact based
  Semantic relationships: Reasoning/workflow
  Provenance: Agent history
  Agent-safe staging: Branch/revert
  Exact external container replay: No
  Authority/completeness courts: Workflow-dependent

- **System:** YoloFS
  Exact content custody: Snapshot dependent
  Versioned project tree: Filesystem state
  Persistent logical identity: Filesystem objects
  Semantic relationships: Limited
  Provenance: Mutation history
  Agent-safe staging: Yes
  Exact external container replay: Filesystem recovery
  Authority/completeness courts: Permission policy

- **System:** Proposed system
  Exact content custody: Yes
  Versioned project tree: Yes

Persistent logical identity: Yes
Semantic relationships: Yes
Provenance: Yes
Agent-safe staging: Yes
Exact external container replay: Yes
Authority/completeness courts: Yes

The proposed system does not introduce each column independently.

Its differentiation is that all columns are part of one correspondence model.

---

16. The Strongest Differentiating Elements

16.1 Exactness and semantics share one substrate

Many systems choose one primary truth:

- exact storage;
- source history;
- backup snapshots;
- semantic indexing;
- provenance;
- agent memory.

The proposed system binds these together.

A file may simultaneously possess:

- exact-byte identity;
- content ancestry;
- path membership;
- semantic role;
- dependency edges;
- derivation history;
- authority standing;
- packaging representations;
- agent-context relevance.

16.2 The filesystem is a projection

The authoritative project is not necessarily the currently visible directory or ZIP file.

The visible filesystem is a materialized view of a deeper project-state graph.

This idea has precedents in Git, OSTree, semantic filesystems, and content-addressed stores. The proposed distinction is that the projection is also:

- semantically indexed;

- authority-governed;
- exactly reconstructible;
- agent-operable;
- format-recursive.

16.3 Same-name successor publication

The stable filename:

`project-latest.zip`

is treated as a visible slot rather than an artifact identity.

Each replacement becomes a new committed generation only after correspondence verification.

This supports a practical AI workflow that conventional source control does not directly model:

1. Receive a complete candidate archive.
2. Compare it against the current project generation.
3. Require complete predecessor disposition.
4. Refuse unexplained omission.
5. Commit new file and content states.
6. Verify the external artifact.
7. Atomically replace the visible slot.
8. Retain all historical states internally.

16.4 Completeness as an admission requirement

Most version-control systems permit deletion as an ordinary state change.

The proposed system can require an explicit explanation for every missing predecessor file.

This is particularly important when a generative agent emits a complete replacement artifact. The system does not assume that absence is intentional.

16.5 Recursive container abstraction

If a project contains historical project archives, the system can recognize them as materializations of states already represented in the same lineage.

Instead of storing a nested archive opaquely, it can represent it through:

- known file states;
- a known project-state reference;
- packaging metadata;
- a compact exact residual.

This generalizes deduplication across representation layers.

16.6 Evidence-class separation

The proposed system explicitly distinguishes:

- reconstructed facts;
- hash-verified documentary claims;
- governed semantic inference;
- externally authenticated attestations.

This prevents a stored document from creating its own authority merely because it is present in a valid package.

16.7 Semantic context compilation

The project graph can generate bounded context for an AI based on:

- target files;
- dependencies;
- relevant historical states;
- associated tests;
- causal findings;
- authority standing;
- unresolved decisions.

This connects storage architecture directly to machine cognition without making the model's memory the sole project authority.

16.8 Failure Modes the Synthesis Must Refuse or Expose

The unification creates failure modes that no individual primitive automatically solves:

- **Hash sufficiency error:** treating byte identity as proof of semantic identity or authority.
- **Disposition laundering:** supplying a syntactically complete but false explanation for a deletion, rename, or derivation.
- **Projection drift:** reconstructing a semantically equivalent tree while failing an exact-container promise.
- **Inference promotion:** promoting a similarity-based file relation to authoritative lineage without adequate evidence.
- **Authority laundering:** allowing a valid package or signed record to manufacture authority it did not possess.
- **Partial-project admission:** accepting a successor whose comparison boundary silently excludes predecessor material.
- **Residual opacity:** using an opaque residual so broadly that semantic decomposition becomes nominal.
- **Context overreach:** compiling agent context that exceeds the authority, privacy, or relevance boundary of the task.

- **Garbage-collection overclaim:** deleting an object because it is unreachable in one projection while it remains required by another lineage or standing claim.
- **Verifier self-trust:** allowing the component proposing or applying a successor to confer its own final standing.

Each failure implies a negative test. A credible implementation must demonstrate refusal or bounded qualification, not merely successful reconstruction on cooperative examples.

---

17. What Is Not Novel

The following should not be presented as original inventions:

- hashing content to derive object identifiers;
- immutable object stores;
- Merkle trees or DAGs;
- content-defined chunking;
- cross-snapshot deduplication;
- binary or semantic deltas;
- file snapshots;
- project trees;
- commits and branches;
- provenance graphs;
- semantic metadata extraction;
- virtual filesystem views;
- staged agent writes;
- filesystem snapshots;
- rollback;
- reproducible derivations;
- stable mutable identities constructed from immutable events.

Any technical claim should acknowledge those precedents directly.

---

18. Plausible Research Contributions

The system may support several narrower, defensible research contributions.

Contribution 1: Governed project correspondence

A formal model establishing correspondence among:

- exact content;
- file-state lineage;
- project-state maps;

- external package bytes;
- workspace projections;
- authority standing.

Contribution 2: Complete successor admission for generative agents

A method by which a complete AI-generated project replacement is admitted only when every predecessor object receives a valid disposition.

Contribution 3: Transtemporal semantic file storage

A storage model in which changed file states may derive from the most economical valid ancestor anywhere in their lineage, while project generations reference persistent file states by identity.

Contribution 4: Recursive lineage-aware container deduplication

A method for recognizing embedded archives or compound files as representations of already-custodied states and storing only representation-specific residuals.

Contribution 5: Evidence-class-aware project verification

A verifier that distinguishes reconstructed facts, documentary metadata, semantic inference, and external attestation in its output standing.

Contribution 6: Unified human and agent workspace projection

A local-first project substrate that materializes ordinary folders, archives, terminals, editor workspaces, and bounded AI contexts from the same governed project graph.

These contributions should be evaluated independently. The architecture does not need every one to be unprecedented for the overall system to be valuable.

---

19. Novelty Thesis

The strongest novelty thesis is:

> Existing systems separately provide content-addressed storage, snapshot history, semantic indexing, provenance, reproducible derivation, data branching, or agent filesystem safety. The proposed system integrates these capabilities into a single local-first project substrate whose authoritative object is a governed semantic-temporal project graph and whose filesystems, archives, workspaces, and agent contexts are exact or bounded projections of that graph.

A second, more operational thesis is:

> The system enables a generative agent to publish complete same-name successor artifacts while an independent substrate preserves predecessor custody, verifies every file disposition, refuses unexplained loss, and commits only a correspondingly valid project generation.

A third thesis concerns representation:

> Exact archive history and semantic file history need not be competing stores. Exact external containers can be represented as governed packaging projections of a persistent logical file lineage, with opaque residuals preserving any representation details not derivable from semantic state.

These are stronger and more defensible than claiming invention of content addressing or version control.

Governed Project-State Lineage Substrate

Across the completed P1 corpus, this paper's non-duplicative contribution is a **governed project-state lineage substrate that binds exact content custody, file-state history, semantic and governance relations, admission, and external projections**. It does not claim that content addressing alone creates project identity, that a filesystem projection is canonical state, or that the prior-art comparison has been externally completed.

- **Adjacent work:** *Identity Spine Doctrine*
  Supplies: Layered identity across artifacts, works, versions, concepts, and systems.
  Project-lineage boundary: The lineage substrate realizes project-state identities without collapsing those layers.
- **Adjacent work:** *Cold Boot Continuity as a Deployment Primitive*
  Supplies: Resumption envelopes and revalidation across runtime loss.
  Project-lineage boundary: A project snapshot can support cold boot; it does not carry live authority or external state by itself.
- **Adjacent work:** *Governed Meaning Base (GMB)*
  Supplies: Governed semantic state, standing, disputes, and revision.
  Project-lineage boundary: GMB can store project semantics; project lineage additionally binds exact file and projection state.
- **Adjacent work:** *Sovereign Software*
  Supplies: Reconstructability, migration, provider removal, and effective exit.
  Project-lineage boundary: Lineage supports reconstruction evidence but does not prove operational sovereignty.

- **Adjacent work:** *Epistemic Lineage Engine*
  Supplies: Source, transformation, decision, and revision lineage for claims and knowledge.
  Project-lineage boundary: Epistemic lineage and project/file lineage interoperate while retaining different subjects and evidence.

A project-state record should bind:

$$P = (project_id, snapshot, parents, file_states, \\ content_objects, semantic_relations, \\ governance_relations, admission, projection_specs, \\ receipts, standing)$$

Exact content identity, file-state identity, project-state identity, logical work identity, named successor identity, and publication identity remain separate. Rename, copy, split, merge, regeneration, and same-name successor relations require explicit evidence. Neither path continuity nor byte identity settles all of them.

An external filesystem, archive, website, or release is a projection under a bound specification. Round-trip reproducibility supports correspondence only for declared preserved properties. Publication is a governed effect with its own authority, completeness, and evidence; a project state does not become published or authoritative merely because it can render a release tree.

The universal GSM can type
`HAS_PROJECT_SNAPSHOT`, `DESCENDS_FROM_SNAPSHOT`,
`CONTAINS_FILE_STATE`, `CONTENT_IDENTICAL_TO`, `RENAMED_FROM`, `COPIED_FROM`,
`SEMANTICALLY_RELATED_TO`, `ADMITTED_AS_SUCCESSOR`, `PROJECTED_AS_FILESYSTEM`,
`PUBLISHED_AS_EFFECT`, `RECONSTRUCTED_FROM`,
and
`HAS_PROJECT_STANDING`. Each edge carries its own evidence class.

This integration establishes four editorial commitments: adopt governed project-state lineage substrate as the official corpus role; adopt the project-state record; preserve six independent identity layers; and treat every filesystem, archive, and publication output as a governed projection rather than canonical state.

Learning-State, Physical-Decision, Semantic-Identity, and Audit Lineage

The fixed-seed comparison with governed continual training, the hydroengineering/geologic profile, The Locus of Meaning, and adversarial alternation shows why project lineage must preserve several different succession claims at once:

$$P^* = (project_id, snapshot, parents, file_states,$$
$$content_objects, semantic_bindings,$$
$$governance_relations, learning_state, decision_state,.$$
$$audit_state, admission, projections, receipts, standing,$$
$$remainder)$$

GCTR creates dataset, retrieval, router, adapter, evaluator, model, and standing successors whose exact predecessors and residual influence must remain reconstructable. The hydroengineering/geologic profile supplies a physical stress case in which sample, batch, passport, site model, design, placement, monitoring, and decommissioning histories cannot be collapsed into file ancestry.

The Locus of Meaning explains why a content hash, pathname, or file-state identity does not settle semantic identity. A governed successor claim must state which binding constitution, jurisdiction, obligations, and consequences are preserved, changed, disputed, or left unresolved.

Adversarial alternation adds audit lineage: each exact subject, finding, reproduction, repair, regression, counterexample, adjudication, and remaining defect must be bound to the campaign generation it actually concerns. A newer clean report does not overwrite an older failure or inherit its standing automatically.

The universal GSM can type
`HAS_LEARNING_STATE_SUCCESSOR`,
`BINDS_PHYSICAL_DECISION_LINEAGE`, `PRESERVES_SEMANTIC_BINDING_CONSTITUTION`,
`CHANGES_BINDING_WITH_REMAINDER`, `AUDITED_IN_CAMPAIGN_GENERATION`,
`REPAIRS_EXACT_SUBJECT`,
`RETAINS_PRIOR_FAILURE`, and
`PROJECTS_WITH_DECLARED_PRESERVATION`.

This randomized review establishes four editorial commitments: extend project-state records with learning, physical-decision, semantic-binding, and audit state; preserve residual influence and off-file physical history; require binding-constitution evidence for semantic succession; and retain exact campaign generations and prior failures across audit repair.

A Multi-Layer Project Identity and Governed Projection Substrate

Fifteen retained downstream uses place project lineage across context compilation, work custody, quant development, admissibility, audit alternation, cold boot, epistemic lineage, continual training, domain standards, legacy migration, MosaiCOS, MCIR topology, physical-decision histories, observability, and implementation codices. They confirm a substrate joining exact content custody to persistent file, project, semantic, governance, learning, physical-decision, and audit lineage without collapsing those identities.

$$P^* = (project_id, generation, parents, file_states,\\ content_objects, path_bindings, semantic_bindings,\\ governance_relations, learning_state,\\ physical_decision_state, audit_state, admission,\\ projection_specs, retention, deletion, receipts,\\ standing, remainder, successor)$$

Byte identity, file-state identity, project-state identity, logical work identity, same-name successor identity, and publication identity are separate. Rename, copy, split, merge, regeneration, deletion, restoration, and generated-artifact relations require evidence. A clean successor cannot overwrite an earlier failure, and source removal cannot erase a dependent semantic or physical history.

Filesystem trees, archives, sites, releases, and publications are governed projections under explicit completeness, preservation, redaction, residual, authority, and round-trip contracts. Reconstructability establishes only the properties actually replayed. Publication remains a separate external effect. The paper's novelty thesis remains provisional until authorized current prior-art and patent review.

The universal GSM can type
`HAS_PROJECT_GENERATION`, `DESCENDS_FROM_PROJECT_STATE`, `BINDS_FILE_STATE`, `RENAMED_WITH_EVIDENCE`, `PRESERVES_SEMANTIC_LINEAGE`, `RETAINS_PHYSICAL_DECISION_HISTORY`, `RETAINS_PRIOR_AUDIT_FAILURE`, `PROJECTS_AS_EXTERNAL_TREE`, `DELETES_UNDER_RETENTION_RULE`, and `PUBLISHES_AS_SEPARATE_EFFECT`.

This review establishes four editorial commitments: adopt the expanded multi-layer project record; preserve six identity layers and off-file state; govern deletion, retention, restoration, completeness, and prior failures; and keep reconstruction, projection, publication, implementation, and novelty standing separate.

20. Falsification Criteria

The novelty thesis should be rejected or narrowed if a prior system is found that already combines all or most of the following:

1. Arbitrary exact content custody
2. Persistent logical file identities
3. File-state ancestry across removal, restoration, rename, and copy
4. Complete project-state maps
5. Exact historical container reconstruction
6. Recursive container decomposition
7. Explicit authority and evidence classes
8. Complete predecessor-disposition admission

9. Same-name agent successor publication
10. Ordinary workspace materialization
11. Semantic context compilation for agents
12. Governed retention and deletion

Finding a system with five or six elements would not eliminate differentiation, but it might significantly narrow the claim.

Exact Project History Is Not Yet Semantic or Political Authority

Public orientation

Governed Semantic Project Lineage joins exact content custody, persistent file and project identities, semantic and governance relations, agent-safe successor admission, and reconstructible external projections. Its differentiation lies in the combined workflow, not a new hash, graph, archive, delta, or version-control primitive. The novelty thesis remains bounded to the reviewed comparison set.

Adversarial tests

1. **Hash-identity collapse.** Byte identity is treated as file identity, logical-work identity, semantic identity, project identity, or publication identity.
2. **Same-name-successor attack.** An agent output with the expected filename silently replaces a predecessor despite missing, split, merged, or semantically changed contents.
3. **Projection-authority attack.** A reconstructed tree, ZIP, site, or release is treated as authoritative merely because it round-trips.
4. **Clean-successor erasure attack.** A later successful generation overwrites failed attempts, audit findings, rejected states, or physical-decision history.
5. **Deletion-and-privacy attack.** Permanent lineage conflicts with lawful deletion, sensitive material, revocation, minimization, or community restrictions.
6. **Availability attack.** A durable identifier or manifest survives while required content, dependencies, keys, resolvers, or interpretation are unavailable.
7. **Novelty-overreach attack.** A reviewed feature combination is presented as universal priority, patent novelty, or absence of obscure and unpublished systems.
8. **Agent-safety attack.** Complete predecessor disposition is checked at the archive boundary while tools, generated files, external state, symlinks, or runtime effects escape it.

Claim and evidence ceiling

The paper offers a plausibly differentiated systems synthesis and a multi-layer project identity model. It does not establish universal novelty, patentability, freedom to operate, complete semantic continuity, publication identity, privacy compliance, safe deletion, or implementation correctness. Reconstruction proves only the properties exercised under the bound projection and environment.

The prior-art and novelty analysis must be refreshed through authorized current primary-source and patent research for a future evidence-qualified revision. Systems claims require an exact implementation, adversarial successor cases, deletion and restoration tests, projection round trips, off-file-state coverage, and scale evaluation. No external research was added in this pass.

Reader-facing consequence

The public benefit is that an evolving project can retain exact ancestry and explain every successor-file disposition without confusing a recoverable archive with the project's meaning, authority, publication status, or right to persist forever.

21. Research and Validation Agenda

21.1 Expand the prior-art corpus

Further review should include:

- patent databases;
- archival and digital-preservation systems;
- event-sourced filesystems;
- persistent object databases;
- artifact registries;
- software-supply-chain ledgers;
- scientific workflow stores;
- model and dataset lineage platforms;
- IDE history systems;
- enterprise content-management systems;
- digital forensics systems.

21.2 Formalize the correspondence model

Define canonical objects and proof obligations for:

- content state;
- file state;
- path binding;
- project state;
- semantic relation;
- packaging projection;

- agent proposal;
- admission decision.

21.3 Build the semantic-first GSOL experiment

Compare:

- adjacent archive deltas;
- global exact ZIP objects;
- file-payload deltas layered over ZIP objects;
- semantic-first file states with exact packaging residuals.

Measure:

- package size;
- object count;
- graph overhead;
- reconstruction time;
- verification time;
- nested-archive reuse;
- copy and rename recognition;
- project-growth explainability.

21.4 Test agent publication

Provide an agent with a stable output name and deliberately induce:

- accidental file omission;
- intentional deletion;
- rename;
- duplicate copy;
- generated-cache inclusion;
- nested prior archive inclusion;
- restoration of an old file;
- modification of a large binary;
- malformed output;
- attempted authority escalation.

The substrate should classify or refuse each case deterministically.

21.5 Compare with YoloFS-style staging

The project should distinguish its proposal from filesystem safety alone by measuring the additional value of:

- semantic lineage;
- exact container reconstruction;
- project completeness;
- context compilation;

- authority standing.

21.6 Realization Funnel and Current Standing

- **Stage:** Possibility
  Project-lineage work: Explore whether exact custody, semantic lineage, and governed agent publication can share one substrate.
  Current status: Demonstrated as a coherent possibility.
- **Stage:** Specification
  Project-lineage work: Define content, file, project, projection, admission, evidence, and standing objects.
  Current status: Candidate specification; this revision makes the core explicit.
- **Stage:** CM1
  Project-lineage work: Enumerate admissible realizations: opaque storage, chunk stores, deltas, recursive containers, centralized or distributed backends, strict or permissive admission policies.
  Current status: Partially enumerated.
- **Stage:** CM2
  Project-lineage work: Prefer realizations that preserve exactness, refuse unexplained loss, bound inference, preserve exit, and separate standing.
  Current status: Criteria proposed, not exhaustively adjudicated.
- **Stage:** Instance Configuration
  Project-lineage work: Choose hashes, schemas, path normalization, evidence classes, limits, policy, and threat model for one deployment.
  Current status: Not specified by this paper.
- **Stage:** Implementation Realization
  Project-lineage work: Build and operate one bounded system.
  Current status: Mosaic Lineage Archive supplies partial predecessor evidence, not the complete proposed system.
- **Stage:** Evidence
  Project-lineage work: Replay artifacts, adversarial admission tests, measure storage and latency, and compare alternative systems.
  Current status: Partial archive evidence; broader claims unvalidated.
- **Stage:** Revision
  Project-lineage work: Change objects, proofs, preferences, and claims while preserving prior lineage.
  Current status: Required after prior-art and implementation results.

The paper is therefore a specification and research-program artifact with a partial empirical predecessor. It is not evidence that the complete architecture already exists or has been independently validated.

22. Conclusion

The proposed Semantic Project Lineage System is not unprecedented at the level of algorithms or elementary data structures.

Its foundations are clearly visible in prior work:

- Venti provides immutable content-addressed archival blocks.
- Borg and restic provide global content-defined deduplication.
- Git provides content-addressed project snapshots and history.
- OSTree provides deployable versioned filesystem trees.
- IPFS provides generalized content-addressed DAGs.
- Perkeep provides stable mutable objects expressed through immutable claims.
- semantic filesystems provide meaning-derived access.
- lakeFS and Pachyderm provide versioned arbitrary data and provenance.
- W3C PROV provides a general provenance model.
- Nix provides derivation-centered reproducibility.
- GCC, AgentGit, YoloFS, and GitLake demonstrate emerging convergence between version control, persistent context, agent workflows, and filesystem safety.

The proposed architecture's significance lies in combining these lines of development around a different central object:

> a governed, exact, semantically navigable project lineage.

Under this model:

- content is stored once;
- files retain identity through time;
- project states are explicit;
- archives are reconstructible projections;
- semantic relationships remain evidence-bounded;
- agent mutations are staged and governed;
- unexplained loss is refused;
- history can be queried, restored, routed, and compiled into context;
- the operator retains ordinary files, folders, archives, Explorer windows, terminals, and editors.

The appropriate novelty claim is therefore neither trivial nor absolute.

It is not:

> No one has previously built content-addressed storage or a semantic filesystem.

It is:

> The reviewed prior art does not appear to unify exact digital custody, semantic file lineage, project-state governance, reconstructible container projections, and safe AI successor publication into one local-first operational substrate.

That proposition is sufficiently differentiated to justify continued formalization, implementation experiments, and a deeper prior-art review. Publication and any patent-oriented conclusion remain separate decisions requiring complete, resolvable sources and appropriate expert review.

The system should presently be described as:

a novel candidate architecture and systems synthesis, constructed from established primitives, with its strongest original contribution residing in the governed correspondence among exact storage, semantic project memory, and agent action.

---

23. References and Review Boundary

Realization Scope and Evidence Boundary

This successor makes the paper's stage boundary explicit. It narrows claims rather than manufacturing missing evidence: a stage named as context is not thereby claimed as implemented, validated, or complete.

- **Cm1.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.
- **Possibility Space.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.
- **Revision.** Existing source passages use this stage only as bounded context or an obligation. The paper does not claim an independently validated contribution at this stage.

Corpus-Wide Reference Positioning

This paper argues that whether the proposed Semantic Project Lineage System represents a genuinely new computing architecture or primarily recombines established mechanisms. Its nearest external comparison fields are storage, crypto, semantic. The sources in [E1], [E2], [E3], [E4], [E5], [E6], [E7] are cited as predecessor literature, adjacent standards, or counterpressure - not as evidence that they caused the Ableman framework or that they validate its novel claims. The comparison isolates where established work already supplies provenance, formal assurance, institutional constraint, empirical method, or engineering practice, and where this paper adds a distinct claim about admissibility, authority, semantic custody, or governed realization.

Within the Ableman corpus, the highest-priority current cross-links are [A1], [A2], [A3], [A4], [A5], [A6], [A7]. These are links among separately custodied works, not silent identity mergers. Their role is to expose mutual dependencies, terminology collisions, and places where a reader should traverse the corpus before treating this paper as self-sufficient.

Reference-pass counterevidence and limit search

The pass applied the following pressure tests: Fixity and preservation can keep bytes stable while representation dependencies, executability, interpretation, and institutional custody decay. A valid hash, signature, or inclusion proof establishes a bounded cryptographic relation, not meaning, authority, safety, or jurisdiction by itself. Formal vocabularies and graph links make commitments explicit but cannot by themselves establish correct identity, shared interpretation, or lossless translation between contexts. These limits are not generic disclaimers: they identify the assumptions that a later empirical, formal, or implementation review must try to break.

Reference-pass influence and novelty boundary

No newly added external source is asserted to be a documented authorial influence unless the predecessor source already said so. `Predecessor` means earlier relevant work;

`adjacent` means a useful comparison; `standard` means an interoperability or assurance baseline; and `counterevidence` marks work that narrows or pressures a claim. Novelty therefore remains a reviewable, claim-level question rather than an inference from a low similarity score or an absent citation.

References

External scholarship, standards, and prior art

[**E1**] Consultative Committee for Space Data Systems. *Reference Model for an Open Archival Information System (OAIS).* CCSDS 650.0-M-3, Issue 3, December 2024. https://ccsds.org/Pubs/650x0m3.pdf **Relationship:** Predecessor preservation model for information packages, representation information, and designated communities.

[**E2**] John Kunze et al. *The BagIt File Packaging Format (V1.0).* RFC 8493, 2018. https://www.rfc-editor.org/rfc/rfc8493 **Relationship:** Adjacent custody and transfer format for payload manifests, fixity, and complete package movement.

[**E3**] Juan Benet. "IPFS - Content Addressed, Versioned, P2P File System." 2014. arXiv:1407.3561. https://arxiv.org/abs/1407.3561 **Relationship:** Predecessor content-addressed network architecture; content identity is compared with, not treated as sufficient for, semantic jurisdiction.

[**E4**] Reproducible Builds Project. *Formal Definition of Reproducible Builds*. reproducible-builds.org source **Relationship:** Adjacent reproducibility practice; byte reproduction is compared with, but not equated to, semantic or constitutional continuity.

[**E5**] Anders Rundgren, Bret Jordan, and Samuel Erdtman. *JSON Canonicalization Scheme (JCS)*. RFC 8785, 2020. https://www.rfc-editor.org/rfc/rfc8785 **Relationship:** Adjacent canonicalization standard relevant to stable cryptographic commitments.

[**E6**] Ben Laurie et al. *Certificate Transparency Version 2.0*. RFC 9162, 2021. https://www.rfc-editor.org/rfc/rfc9162 **Relationship:** Adjacent append-only Merkle-log standard for inclusion and consistency evidence.

[**E7**] Manu Sporny et al., eds. *Decentralized Identifiers (DIDs) v1.0*. W3C Recommendation, 19 July 2022. https://www.w3.org/TR/did-core/ **Relationship:** Adjacent decentralized identity standard; identifier resolution is not assumed to settle semantic standing or jurisdiction.

Ableman corpus

[**A1**] Adam Ableman. *Constitutional Storage*. Ableman Corpus current edition; work identity `PAPER.PUB.20788662`;
source SHA-256
`115725C7F7B72C3B720E4EF67F660F329B778D34C4D4232848C6082E849203F7`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A2**] Adam Ableman. *Cold Boot Continuity as a Deployment Primitive*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-3DB3470715D914D0`;
source SHA-256
`4CBB05E90E4ECA37EDA78C0E1FC1FDD180B2D6E0936831B57379AD7124776D4E`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A3**] Adam Ableman. *Semantic Custody*. Ableman Corpus current edition; work identity `PAPER.PUB.20300952`;
source SHA-256
`E3887FE98E6369E39AC3FC2EAC63F2DF7C2AD18AD682B3A745C0CD0AD241D0F0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A4**] Adam Ableman. *The Implementation Codex*. Ableman Corpus current edition; work identity
`AMCS-LOCAL-WORK-872EEF9E301B18DE`;
source SHA-256
`1373E9770B5B86EB394BFAF993F5526155D406F4528694B465FDE22A7D525132`.

Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A5**] Adam Ableman. *The Three Hash Rule.* Ableman Corpus current edition; work identity `PAPER.PUB.20788617`;
source SHA-256
`F6FA73D083B70711A8C817F7779751D1506FEBA0E356DB9805BB1570663731B5`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A6**] Adam Ableman. *Constitutional Semantic Hashing: Governed Commitment Schemes for Semantic Provenance, Admissibility, and Constitutional Continuity.* Ableman Corpus current edition; work identity `PAPER.PUB.20248734`;
source SHA-256
`B1DAC3E2CD4D57BAFBF6462BA6ACFFF7CF120CDB7FA37D0C173525744CC0A2F0`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

[**A7**] Adam Ableman. *Cryptographic Admissibility and Jurisdiction.* Ableman Corpus current edition; work identity `PAPER.PUB.20602448`;
source SHA-256
`6094338D8706F63E8787A5E8DCF40F8A651C0FC3E5EA776DD61791566D094D38`.
Corpus relationship: priority cross-link selected by corpus profile and similarity review; no identity merger or publication effect.

Editorial source notes retained from predecessor

Adam Ableman. *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. https://doi.org/10.5281/zenodo.18453681.

Adam Ableman. *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality.* Zenodo, 2026. https://doi.org/10.5281/zenodo.18601631.

Adam Ableman. *Governed Context & Memory (GCM): Context Configuration as Constitutional Surface, Semantic Hashing as Enforcement, and Explicit Ignorance as First-Class Success in Generational AI.* Zenodo, 2026. https://doi.org/10.5281/zenodo.18633797.

Adam Ableman. *Semantic Custody.* Zenodo, 2026. https://doi.org/10.5281/zenodo.20300952.

Adam Ableman. *The Three Hash Rule.* Zenodo, 2026. https://doi.org/10.5281/zenodo.20788617.

Adam Ableman. *Constitutional Storage.* Zenodo, 2026. https://doi.org/10.5281/zenodo.20788662.

Adam Ableman. *Reasoning by Way of Receipts.* Ableman Corpus current edition; conceptual claims retain the standing stated in the work.

External prior-art boundary

This draft names Venti, restic, Borg, Git, OSTree, IPFS, Perkeep, semantic filesystems, lakeFS, Pachyderm, W3C PROV, Nix, Git-Context-Controller, AgentGit, YoloFS, and GitLake. A publication candidate must add a resolvable primary-source reference for every comparison and recheck the feature matrix against those exact versions. This pass does not invent bibliographic metadata or convert a remembered comparison into verified novelty evidence.

The entries below are corpus-internal comparison and dependency sources. Their inclusion does not establish external validation of this paper's empirical, safety, novelty, or performance claims.

Adam Ableman *Governed Semantic Compilation: Lawful Continuation, Refusal Semantics, and Epistemic Promotion in Large Language Models.* Zenodo, 2026. doi:10.5281/zenodo.18453681

Adam Ableman *The Three Hash Rule.* Zenodo, 2026. doi:10.5281/zenodo.20788617

Adam Ableman *Constitutional Storage.* Zenodo, 2026. doi:10.5281/zenodo.20788662

Adam Ableman *Governed Context & Memory (GCM): Context Configuration as Constitutional Surface, Semantic Hashing as Enforcement, and Explicit Ignorance as First-Class Success in Generational AI.* Zenodo, 2026. doi:10.5281/zenodo.18633797

Adam Ableman *Receipt-Bound Capabilities: Membrane Tokens for Governed Interaction with Reality.* Zenodo, 2026. doi:10.5281/zenodo.18601631

Adam Ableman *Semantic Custody.* Zenodo, 2026. doi:10.5281/zenodo.20300952

Adam Ableman *CARA: Constrained Artificial Reasoning Architecture.* Zenodo, 2025. doi:10.5281/zenodo.18099715

Adam Ableman *A Theory of Agentic Observation.* Zenodo, 2026. doi:10.5281/zenodo.20437813

Corpus source and companion formats: https://github.com/AblemanMosaic/ableman-corpus
License: CC-BY-ND-4.0

DOI: 10.5281/zenodo.23847752